

From netlist to verified model

Automatic real-number-model and testbench generation
for mixed-signal verification

Veylon Systems

Abstract

A real number model is a fast stand-in for an analog block. Rather than solving transistor physics the way SPICE does, it computes an output number from an input number — so it runs in an ordinary digital simulator, alongside the chip’s logic, fast enough for a full regression suite.

The trouble is that somebody has to write it, keep it matching the circuit as the circuit changes, and convince the team it still matches. The first two are slow. The third usually never happens at all.

spice2rnm does all three from one command. Give it a SPICE netlist and you get back a SystemVerilog real number model, a UVM-MS verification environment that exercises it, and the evidence that both agree with the real circuit.

Three things make that evidence worth reading. Every expected value in every generated testbench is a *SPICE measurement of your circuit*, never the model’s own equations, so a test cannot pass by agreeing with itself. Every generated environment is proven able to fail: the tool breaks the model on purpose and confirms the test goes red. And how a block gets checked depends on what that block is for — a filter as a frequency response, a duty-cycle corrector by where its edges land, a phase interpolator by how far it moves the clock at each setting.

This paper walks through four circuits, getting harder as it goes, and runs the last of them twice — once against a co-simulation, once with none at all. Every number in it was produced by the shipped tool, in about an hour on a laptop.

On one page

What you hand it. A SPICE netlist — flat, or with subcircuits — and optionally a plain-language specification and an existing model library to take conventions from. No port annotations, no model template, no testbench.

What you get back, from one command. A SystemVerilog real-number model with the structure chosen from the measured circuit; a UVM-MS environment that exercises it, with every expected value a SPICE measurement of your circuit; and the equivalence evidence — the circuit and the model driven identically, compared in the domain that matters for that block, traces archived.

What it produced on five circuits, each one command, about an hour in total on a laptop, on open-source simulators:

Circuit	Judged on	Worst, against its bar	Environment
two-pole filter	waveform, unseen input	0.0097 vs 0.15	64 checks, 0 failed
duty-cycle corrector	where its edges land	0.428 vs 1.0 pp	60 checks, 0 failed
phase interpolator	edge position per code	1.83 vs 6.25 ps	9 checks, 0 failed
PLL, co-simulated	system vs transistors	4.62 mV, 12% of bar	5 blocks, 0 failed
PLL, no co-simulation	loop closed in SV alone	locked, +0.0004%	12 checks, 0 failed

Why a green result here is worth believing. Three rules, kept throughout.

- The right answers come from SPICE, never from the model. A test cannot pass by agreeing with itself.
- Every environment is proven able to fail. The tool breaks the model on purpose and confirms the test goes red, and measures how small a break it still catches.
- Each block is scored on its own job — a filter on its response, a clock buffer on its edges, an oscillator on its rate — and a check that does not apply is reported as not applicable, never counted as a pass.

Where the evidence is. Every model, environment and verdict in this document is published exactly as the tool wrote it, at <https://github.com/vrajeshprakhya/spice2rnm-demos>, and regenerates from one command. The outputs are plain IEEE 1800 SystemVerilog and carry no vendor dependence; they ran here on ngspice and on the open-source simulator xezim, and will run on the simulators your flow already uses.

Contents

1	The problem	3
2	What you get from one command	4
3	Case study 1 — Two-pole filter	5
4	Case study 2 — Duty-cycle corrector	6
4.1	A model that follows the circuit’s dynamics	7
4.2	Verified by function, not by formula	7
5	Case study 3 — Phase interpolator	7
6	Case study 4 — Phase-locked loop	8
6.1	Checked as a system, against the transistors	9
6.2	The rail the deck declares is the rail the model sees	10
6.3	Where the block works is measured, not assumed	11
6.4	The same loop, with no co-simulation to probe it	12
7	Results at a glance	16
7.1	What the equivalence folders hold	18
8	What the equivalence check compares	19
8.1	What keeps a figure from flattering itself	19
8.2	Blocks, and then the system	20
8.3	One thing a score at an operating point cannot see	20
9	Why a green testbench is worth believing	22
10	Functional specifications as input	22
11	Joining an existing model library	23
12	AI where it helps, physics where it decides	24
13	Scope and requirements	24
A	What the routes measure	26
A.1	Why an oscillator is compared as a rate	26
A.2	The tuning curve	27
A.3	Control states	27
A.4	The state space	28

1 The problem

A model of an analog block has to be two things at once, and they pull against each other. Fast, or it cannot run in a digital regression suite at all. Accurate, or passing it does not mean anything.

Hand-written models drift away from the circuits they stand for, and the drift is silent. Nothing breaks. The model still compiles, still simulates, still passes its tests. It has simply stopped describing the circuit.

The usual defence is a hand-written testbench, which has the same problem one level up. A person decides what to check and how close is close enough, and those decisions are rarely written down beside the numbers they produced. When the circuit is revised, the model, the testbench and the tolerances all need revisiting by hand — which, in practice, is not what happens.

spice2rnm regenerates all three from the netlist whenever you ask it to. The model, the environment that checks it, and the evidence trail all come out of the same command, so they cannot quietly fall out of step with the circuit or with each other.

2 What you get from one command

```
| spice2rnm dcc.cir --emit-uvm-ms --spec dcc_spec.md
```

The ports are not named because the tool reads them. The input is the node driven by the one source carrying a transient function or an AC magnitude, where the deck's supplies carry a DC level and nothing else — on this netlist, `clk`.

The output is a different kind of question. How a circuit is wired does not say which node you care about. What does say it is the deck's own `.save`, `.print` or `.probe`: that is the author naming the signal the netlist exists to show, and where a deck carries one the tool takes the output from it. Where a deck carries none, as this one does, the output is a guess from a node's name. The run says which of the two it used, so a reader can tell a declaration from a guess. Four routes decline to guess at all and ask to be told — the ones measuring a particular node's edge, frequency or current.

- **A SystemVerilog real-number model**, with the model structure chosen automatically to match the block's measured behaviour — including blocks whose dynamics move with signal level, where any fixed-response model is wrong across most of its range.

The model presents `wreal` ports, so it wires straight into an RNM library like the models beside it — and it is the model itself that does, not a wrapper around it.

That distinction is functional, not stylistic. A `real` port is a *variable*, and a variable takes exactly one driver: nothing can share the node and no bridge can attach without a converting module in between. A `wreal` port is a *net*, so drivers combine on it — an offset source, a noise injector, a second bridge — and the model drops into a library with nothing standing between it and the nets it is wired to.

It costs one thing, and the cost is real: `wreal` is Verilog-AMS vocabulary rather than IEEE 1800, so the model compiles where the simulator provides it, and everything instantiating the model inherits that. For a flow without it, `--model-ports real` emits the same model with variable ports, depending on no package, no nettype and no extension. `--model-ports house` gives it your own library's nettype instead (Section 11).

The computation is identical in every case — only the declaration of the ports differs, and a net cannot be assigned procedurally, so a net-typed model drives its output through one continuous assignment. Measured rather than asserted: the filter scores `rms_error_norm` 0.029008790476737805 against SPICE with either discipline, the same to every digit.

- **A UVM-MS verification environment** following the Accellera UVM-MS specification, built on IEEE 1800 — not on Verilog-AMS. Analog values travel on *user-defined nettypes* (IEEE 1800-2017 §6.6.7) with a generated resolution function: the emitted package declares the nettype together with a `sum_res` that adds every driver's contribution. Multiple drivers on a net then combine in the net type rather than in hand-edited testbench code — an offset source, a noise injector, a second bridge. The environment instantiates the model itself rather

than a copy of it behind a wrapper, so what the checks cover is what ships. Its own structure is IEEE 1800 throughout — the nettype, the resolution function, the `interconnect` ports — but the model it drives carries `wreal`, so running it needs a simulator that provides it. Bridge and wrapper ports are `interconnect` (§6.6.8), inheriting the testbench’s nettype instead of naming one, and scoreboard golden values are SPICE measurements of your circuit.

- **Equivalence evidence:** circuit and model driven with the identical stimulus, then compared in the domain that matters for the block — waveform error for signal-path blocks, edge placement for clock-path blocks. The aligned traces are archived, so any verdict can be re-derived from its own run directory.

An optional plain-language functional specification supplies what no netlist can: the block’s intended operating range and its acceptance criteria (Section 10).

3 Case study 1 — Two-pole filter

A two-pole RC low-pass filter — the ordinary case, a block that takes a signal in and shapes it on the way out. This is the whole input, its comment lines aside:

```
2-pole passive RC low-pass filter -- fast-running RNM test case
VIN vin 0 DC 0.9 AC 1
R1 vin mid 1k
C1 mid 0 15.9p
R2 mid vout 1k
C2 vout 0 15.9p
.end
```

Nothing else: no directives naming the ports, no hint about what the block is for, no declared corner frequency. The generated model is checked four independent ways.

Chirp equivalence	0.0290 against a 0.15 threshold	PASS
PRBS (independent stimulus)	0.0097 normalised	PASS
UVM-MS, DC section	49 checks, 0 failed	
UVM-MS, AC section	15 checks, 0 failed	
worst magnitude	0.0674 dB of a 0.25 dB tolerance	
worst phase	2.70° of an 8.00° tolerance	

This is what it produced — the head of `rc_lpf2_rnm.sv`, as published:

```
// SystemVerilog Real Number Model, auto-generated by spice2rnm.
// Fitted transfer function: H(s) = 1 * 1 / ((1+s/2.4023e+07) * (1+s/1.64656e
+08))
// poles (Hz): [3823382.3097665044, 26205852.219306976]
// fit quality: rms_error_db=1.417e-08 max_error_db=5.81e-08

module rc_lpf2_rnm(input wreal in_val, output wreal out_val);
```

The poles are measured, not declared, and the model carries the quality of its own fit at the top of the file. A reader who wants to know how good it is does not have to run anything.

It also says where it stops being evidence. Every model carries the range it was characterised over, and an input outside that range is reported rather than quietly served:

```
localparam real DOMAIN_LO = 0.0;
localparam real DOMAIN_HI = 1.8;

a_input_domain: assert (in_val >= DOMAIN_LO - DOMAIN_EPS
```

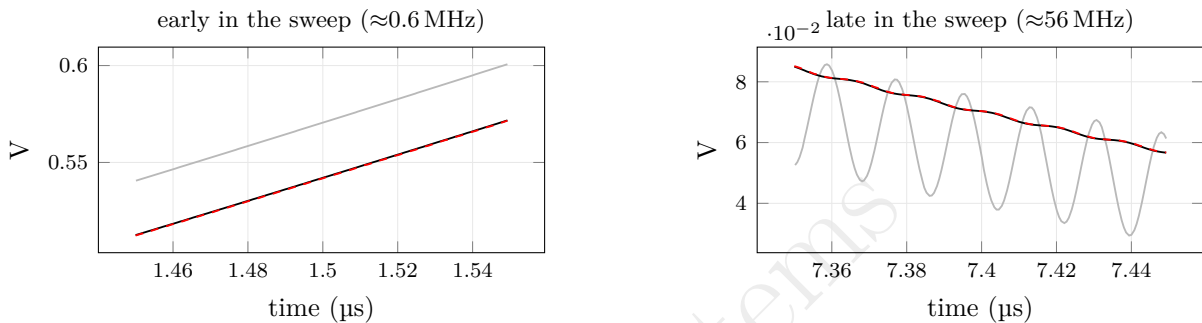
```

    && in_val <= DOMAIN_HI + DOMAIN_EPS)
else ... $warning("%m: input %0.6g V is outside the characterized
    range %0.6g..%0.6g V. static_curve clamps there, so this model
    keeps running and returns the endpoint value -- it is not
    extrapolating, and it is no longer evidence. Reported once.");

```

A model asked for a voltage it was never characterised at does not fail and does not extrapolate — it clamps, which means it keeps returning a plausible number. That is the dangerous case, and it is the reason the warning exists. Once per model, so a long run stays readable.

The PRBS row matters more than it looks. The model is built from a chirp — a tone sweeping upward through frequency — and then tested with a pseudo-random bit sequence it has never seen. A model that had merely memorised the input it was built from would come apart here. This one does not.



Grey is the stimulus, black the transistors, red dashed the generated model — two moments of the same chirp, at the simulator's own resolution. Early, the output follows the input. Late, the input still swings and the output barely moves: that is the filter. Measured over 100ns windows the output-to-input swing ratio is 0.99 in the first window and 0.50 in the second, and the model tracks the transistors through both. Drawn from `waveform_compare.csv` and the PWL stimulus in that run's own `equivalence/` directory.

4 Case study 2 — Duty-cycle corrector

A clock is meant to be high for half of each cycle. When it is not, a duty-cycle corrector fixes it. This one is built around a comparator.

Ask how fast the block responds and you get two answers. Nudge the input a little and watch the output move, and the answer is 3.7×10^8 rad/s. Swing the input hard and time the output with a stopwatch, and it is between 1.3×10^{10} and 5.1×10^{10} rad/s — 35 to 137 times faster.

The slow answer is not wrong. It measures a delicate node inside the comparator, and sensitive things are sluggish. But the output does not come from that node. It comes from a buffer that slams between the rails and does not wait. So the slow number is true about something the output ignores, and a model built on it runs tens to hundreds of times too slow.

The faster answer is not a single number either. It moves by a factor of 3.9 across the input range, so a model with one fixed speed cannot follow it.

Then the trap. The usual way to check a model like this compares its AC response against the circuit's. Both are measured by nudging, so both report the slow node, and the two agree. The check passes on a model that is a hundred times too slow, and it has said nothing about the error.

That is why blocks like this tend to have the hand-written models people trust least.

4.1 A model that follows the circuit’s dynamics

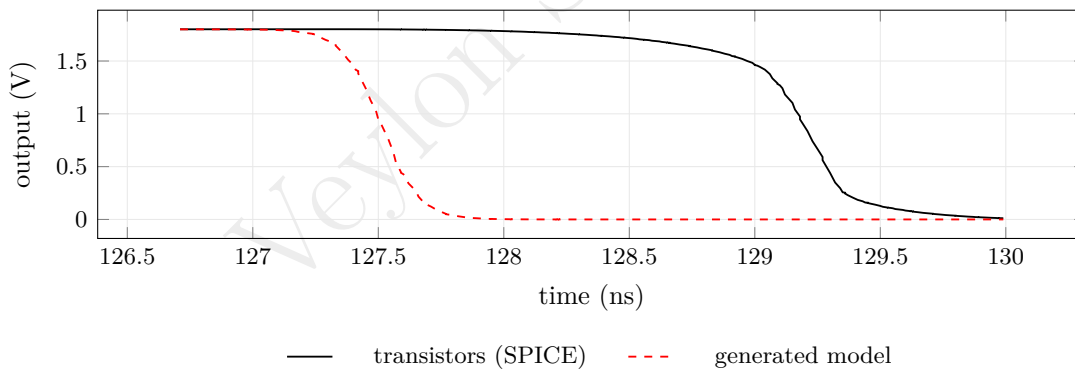
spice2rnm notices that level dependence while it is measuring the circuit, and builds a model whose speed varies with level the way the measurements say it should. That includes the circuit’s rise/fall asymmetry — it takes longer to switch one way than the other, by a measured 95 ps — which the model reproduces. Fed a clock that is high exactly half the time, the model’s output duty matches the circuit’s to 0.000 percentage points.

Transient equivalence	0.0769 against a 0.15 threshold	PASS
UVM-MS, DC section	46 checks, 0 failed	
UVM-MS, duty section	14 checks, 0 failed	
worst duty error	0.428 pp of the specification’s 1.0 pp	

4.2 Verified by function, not by formula

The testbench generated for this block has *no AC section*, and that is a measured decision rather than an oversight. An AC sweep assumes the circuit sits at one operating point and wiggles gently around it. A comparator in normal use never does that — it slams between two states. Check this model that way and it fails every single point by more than 55 dB, despite being demonstrably accurate. The failure tells you nothing about the model and everything about using the wrong check.

So the environment checks what a duty-cycle corrector is actually for. The tool drives the real circuit with clocks at two rates and three input duties — the rates and the 35–65% range its specification states (Section 10) — records what duty comes back out of SPICE each time, and holds the model to those recorded answers. The table spans 30 percentage points of measured output duty — wide enough that a model ignoring its input could not fake its way through.



One falling edge at the simulator’s own resolution. The model places it 1.68 ns early, and places the next edge 1.60 ns early too, so the interval between them — the duty this block is specified on — is preserved to 0.03%. An average voltage error over the whole run would report this as a large disagreement; the duty check the environment actually applies does not.

5 Case study 3 — Phase interpolator

A phase interpolator does not change a signal’s size. It shifts the signal in *time*: each control code slides the clock edge a little further along. So there is no frequency response worth fitting, and comparing voltages just measures noise. Conventional flows have nothing to offer this block.

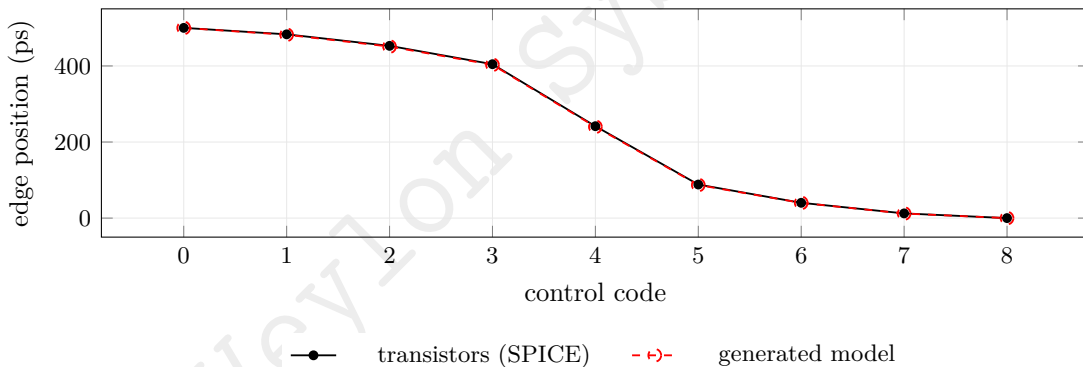
spice2rnm recognises what the block does and changes approach. It measures how far the edge moves at each code, builds a delay-line model, and generates a *phase* testbench — one measured answer per control code, and no DC or AC section, because neither of those describes where an edge lands.

Codes measured	9 of 9
Total traversal	−500.1 ps, monotonic
Mean step (1 LSB)	−62.51 ps
Worst DNL	−100.5 ps (−1.61 LSB)
UVM-MS phase section	9 checks, 0 failed
worst	3.26 ps of a 6.25 ps tolerance

Two terms from that table. **One LSB** is one step of the control code — the nine codes span 500.1 ps in eight steps, so a step is worth 62.51 ps, and that is the natural unit for everything else here. **DNL** is how wrong each individual step is against that average.

This dial is lumpy. Differencing the measured edge positions gives eight steps of −17, −30, −48, −163, −153, −48, −28 and −12 ps, where every one of them should be −62.5. The worst is that fourth step: it moves 163 ps for one click of the code, 100.5 ps more than it should, which is the −1.61 LSB in the table. A block like this usually sits in a loop that nudges a sampling clock one code at a time to stay centred in a data eye, and such a loop assumes a click is worth about one LSB. Here one click moves 12 ps and another moves 163 ps, so the loop’s gain varies by more than 13× depending where it starts.

The tolerance is a tenth of an LSB, the finest the model can resolve — tight enough to catch a real error, loose enough never to trip on rounding. All of that lumpiness is the circuit’s own imperfection, reported as measured. The model’s job is to reproduce it, and it does: a model that drew an evenly spaced line would look tidier and would be hiding a real defect from whoever verified against it.



Nine codes, nine measured answers. There is no waveform to compare here: what the block does is place an edge, so that is what is plotted.

6 Case study 4 — Phase-locked loop

The first three are single blocks. This one is a *loop*, and that changes what a model has to get right. In a loop, a small error does not stay small — it is fed back around and pushes the whole system to a different operating point. So a model that looks perfectly reasonable on its own can still settle the loop somewhere the real circuit never goes.

It is also three different characterisation problems in one netlist, and no single method covers them. The deck’s own `.subckt` boundaries are the blocks:

```
.subckt ro_vco aout cont dd
...
xin1 dd ss sub well cont in1 in1 out1 out1 gaincell
...
xin11 dd 0 sub well out1 out2 inv1
xin12 dd 0 sub well out2 aout inv2
```

```
.ends ro_vco

xcp  pupb pdn dra dd  cpump
xlf  dra vout          lpfilt
xvco aout vout dd     ro_vco
```

Three `x` cards at the top level, so three models are composed into the runnable whole. `inv1` and `inv2` are instantiated one level down, inside `ro_vco`: they get their own models and their own verdicts — which is why the score below is five of five and not three of three — but they are not wired into the composed top, because the `ro_vco` model was measured with them in it already. `gaincell` gets no model at all: two inputs and two outputs, so no route that fits one input to one output can characterise it, and it does not need one.

The pipeline classifies each remaining block from its topology and routes it accordingly:

Block	Why the default does not fit	Route taken
<code>ro_vco</code>	oscillates with its inputs at DC — its output is a frequency, not a gain	tuning curve, scored on rate
<code>cpump</code>	a complementary pair driven from independently controlled gates	one $I(V)$ table per control state
<code>lpfilt</code>	only R and C, so its state space is exact and needs no fitting	exact state space, read from topology

Not one of those three is a simple input-to-output response. A conventional flow that fitted one to each would be fitting the wrong quantity three times over.

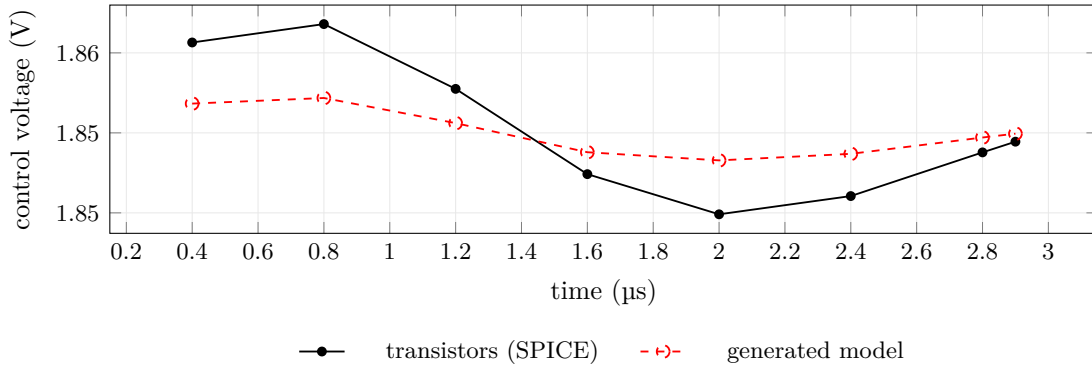
What a tuning curve, a control state and a state space actually *are* — and why an oscillator is compared as a rate rather than as a waveform — is set out in Appendix A, with this circuit's own numbers.

6.1 Checked as a system, against the transistors

A score for each block on its own is evidence about that block on its own. A loop asks a different question: wired together, do these models behave like the circuit? The co-simulation answers it directly. The design's own testbench is run twice: once where the analog half is the generated models, once where it is the real transistors through ngspice. Only the small piece of code that connects the two halves is swapped between runs. The thresholds, the expressions and the pass/fail logic are byte-for-byte identical in both, so the two logs can be compared line against line.

Blocks modelled, three routes	3 composed, 2 more emitted
Per-block equivalence	5 of 5 pass
Supply condition	20 mV rms, declared in the deck
System check vs transistors	4 boundary signals, 0 failed
vout (control voltage)	4.62 mV, 12.4% of its bar
aout (VCO output)	0.4% of crossings, bar 2%
dra (internal probe)	20.3 mV, 0.9% of its range
Both loops lock at	400.00 MHz
control band, transistors	1.8449–1.8568 V
control band, models	1.8483–1.8522 V
Model half vs transistor half	6.36 s vs 105.3 s
same testbench, same span	17×

The control voltage is the number to read. It is where the loop stores its state, so a model that settles anywhere else disagrees here first — and the two agree to 4.62 mV while reaching the same frequency, on a rail carrying 20 mV rms of noise the deck declares. That is looser than the figure this table carried while the rail was quiet, and the rail is the reason: a loop concentrates a supply disturbance onto precisely this node. Running the case that way is the point of it.



The control voltage at the co-simulation boundary: the composed models against the transistors, on the design's own testbench, on a node the loop holds near 1.85 V. Both settle to the same place and hold it. Eight samples because that is how often the digital side reads this node; how closely they agree at any one instant is a draw, and the table's bar is what the verdict is taken against.

That figure is one draw. The bar beside it is not.

Both runs make their own noise from the source the deck declares, so neither ever sees the same noise twice. The worst-window difference therefore changes every time. Seven consecutive runs of this same deck gave 3.61, 1.83, 4.15, 2.05, 1.86, 1.72 and 4.62 mV, and the last two were a pair run back to back with nothing changed between them. So that figure is a sample of a distribution, not a property of the model.

What does not change from run to run is what the difference is compared against. That bar is 0.265 of `vout`'s own scale, and 0.215 of it exists to absorb exactly this variation. The 4.62 mV above comes to 0.124 against it — comfortably inside. That is the number the verdict is taken on, and the number the record keeps.

The scale has a floor of its own: 2% of the node's magnitude. Without one the comparison would be unfair here, because the transistor run's `vout` moved only 11.8 mV across the window, and judging a difference against so small a range would report ordinary noise as error.

The general point is worth stating plainly. An agreement figure quoted without its bar tells you less than it appears to — in the same way a jitter ratio means little without the correlation time it was measured at.

Each node also records which metric its bar applies to. Some of them the digital side reads through a threshold, where what matters is when the signal crosses and not what it averages; those are judged on their crossings. Printing the average next to a crossing bar would put a pass beside two numbers that contradict it — 0.030 against a 0.020 bar — when the figure actually compared was 0.004.

6.2 The rail the deck declares is the rail the model sees

The netlist for this case says

```
vdd dd 0 dc {vcc} trnoise(0.02 1e-10 0 0)
```

— a supply with 20 mV rms on it, redrawn every 100 ps. A composition that reads that line and emits `assign dd_v = 3.3;` has deleted a disturbance the design asked for, and every check downstream then agrees about a circuit nobody is shipping. What the tool emits instead, in `netlist_top.sv`:

```

int unsigned dd_rng;
initial dd_rng = DD_SEED;
real dd_nw;    // white, sample-and-hold at the deck's NT
initial begin
    dd_nw = 0.0;
    forever begin
        #(DD_NOISE_DT_S * 1.0e15);
        dd_nw = DD_NOISE_RMS * __gauss(dd_rng);
    end
end
assign dd_v = ((dd_v_ext > 0.0) ? dd_v_ext : DD_NOM) + dd_nw;

```

The amplitude and the redraw interval are the deck's own, the rail stays overridable from outside, and the draw is seeded so the run repeats. A transient of this deck puts that source at 10.91 ps rms of period jitter at the oscillator, $\pm 5\%$ at the two hundred cycles it was measured over. That figure is itself a draw. A constant rail produces none of it.

That figure is measured per design rather than quoted. Getting it needs one step first: the declared condition is restored onto the block's own supply. Extraction rebuilds every rail as a plain DC source, so the fragment a block is characterised from does not carry the noise its design states. Without that step the measurement would be of a quieter circuit than the one being modelled. The generated file says which source the number came from.

So the declared source is carried into the composed model, sampled and held at the source's own interval. That reproduces the two properties which decide how much jitter a source causes — its rms and its correlation time. The spectra agree at low frequency and part near $1/NT$, where ngspice interpolates between its noise breakpoints and the model does not.

All seven fields of a `trnoise` declaration are read, not the white pair alone. Where a deck declares flicker, the composition carries it as a Voss–McCartney summation across the octaves the run spans; where it declares a random telegraph, as a two-state Markov process with exponentially distributed dwells, at the capture and emission times given. The Voss construction produces $\alpha = 1$, and the generated file records both that and the exponent the deck asked for rather than quietly substituting one for the other. This PLL declares the white pair alone, which is why every figure here is white — $1/f$ is what dominates close-in phase noise, and a deck that states it gets it.

Carrying it changes what a comparison means. Both halves now draw their own noise, so two runs of the same circuit differ by sampling error alone and a fixed tolerance would be measuring run length. Each node's bar is widened by three standard errors of the difference between two window means, taken from the reference's own spread rather than from the declared amplitude — what actually reaches a node is a property of the circuit. On `vout` that allowance is 0.215 of the 0.265 bar, and the record carries both, so a reader can see how much of the bar was earned and how much was allowed.

6.3 Where the block works is measured, not assumed

The two output-buffer inverters sit in the clock path. Their job is to place edges in time, and an average voltage error is close to blind to that. Both pass an average-error comparison comfortably — while their error at the single worst instant is 99.9% of the full voltage swing. Those are not in conflict: if the model's edge lands slightly late, then for that brief moment the model sits at one supply rail while the real circuit sits at the other. Averaged over time it vanishes. For a clock buffer, that moment is the behaviour being verified. The pipeline classifies them from the measured DC curve — 62% and 61% of swept points within 5% of a rail is a switching characteristic, not a smooth response — and scores them on duty cycle instead.

The frequency it scores them at is measured. Where a block operates is a property of the closed loop and exists only once the digital side is attached, exactly like the operating range the golden probe already measures. The probe therefore measures each net's switching rate beside

its voltage band: the two inverter inputs return 399.8 and 399.7 MHz, the 400 MHz this PLL locks to. Scored there, they pass at 0.276 and 0.307 pp of duty against a 0.5 pp bar.

The form they are re-emitted in has no delay line, so it cannot carry the measured rise/fall skew — 0.589 and 0.672 pp of duty at that clock. Both blocks measure below their own floor, so the skew partly cancels rather than adding. That makes the floor an upper bound on what the form costs, not margin to spend, and the pipeline states it before scoring rather than leaving a pass to be read as unqualified.

That number is not one a language model can supply. Asked five times about the same inverter with the same measurements, the classifier proposed clock rates of 2 MHz, 765 MHz, 300 MHz, 10 GHz and 15 GHz; asked about the second stage, it proposed 10 GHz. The lowest is two hundred times below where the buffer runs and the highest thirty-seven times above what its own model can follow. The measurement returns the same answer every run, and gives the two stages answers agreeing to 0.03%, which no classifier run did. Intent from the model, physics from the circuit, applied to a number that looked like intent and was not.

What the classifier is asked for, it has been steady on: buffer, clock stimulus, duty metric, on every run of both stages. That is the judgement gated on a measurement it can check itself against. What moved is the one number it was in no position to know.

The verdict names the metric it came from. Asking for the classifier is not the same as getting it. Every duty figure above comes from a run with a key present and the classifier consulted. The same demonstration was then run deliberately *without* one, to see what a reader would be shown if it declined. The chirp is kept, and these same two blocks pass an rms comparison at 0.025 against a 0.15 bar — a green result from a measurement that cannot see the property in question. The two transcripts are otherwise alike, and the command line is identical, so the run records which figure each verdict was reached by, and the demonstration reports it from that record. Asking for a check on the command line does not show that it ran.

6.4 The same loop, with no co-simulation to probe it

Everything above this point in this case study needs a co-simulation to already exist. The golden probe runs the design's own testbench against the transistors; the system check runs that testbench twice. Both need a testbench spanning the two halves, a DPI bridge, and a shared `libngspice` for it to load.

Most designs have none of those. What they have is an analog netlist, some RTL, a specification in English, and nothing wiring the three together. The same PLL was run that way, and two things break.

The operating band has nothing to measure it. Where a block works is a property of the closed loop, and with the loop open there is no loop to measure. Worse, two of the ring's internal nets are driven by nothing at all and have no operating point either. So the band is *declared*, from the table a designer writes anyway — control voltage 1.75–1.95 V, supply 3.3 V, buffers full-swing between the rails, substrate and well tied:

```
--band vout=1.75:1.95      the control window, from the table
--band xvco.outm1=0:3.3    rail to rail, inside the ring
--band xvco.sub=0:0        substrate tie
```

And the tool reads that specification. The demonstration hands it the document along with the netlist and the RTL. What it took from this one, in the run these figures come from:

```
read as: Document states "PLL -- 400 MHz ring-oscillator
phase-locked loop" with "seven-stage differential ring" whose
"behaviour is a frequency, not a gain" -> oscillator/vco
```

```
environment; "Output ... 400 MHz" is the highest stated
frequency; "Reference ... 10 MHz" is the lowest stated rate;
"20 mV rms of wideband noise is specified on it"; "hold 400 MHz
to within 2%" and "No small-signal specification ... Do not
verify any of them against a frequency response" implies
timing-based verdict; control-voltage swing range quoted as
"characterizable from 0.5 V to 2.8 V".
```

Those are the same facts the command line states, arrived at independently, and Section 10 describes the rule that governs them: every extracted fact is checked against what the circuit was measured to do, and the measurement wins. Here nothing disagreed.

More useful is what it refused to let pass. The document names 20 mV rms of supply noise as a condition the part must work under. The reader placed that correctly — an operating condition belongs in the stimulus, not in the model — and then reported that the generated environment does not drive it:

```
spec: the document states 20 mV rms of supply noise as a condition
to work under. That is a property of this block's environment, so
it belongs in the stimulus rather than in the model
NOTE: the supply-noise condition is not driven by the generated
environment yet; the composition carries only the noise the
netlist itself declares
```

A requirement it cannot yet meet is reported rather than dropped. That is the behaviour worth having from a language model in a verification flow: not a proposal, a gap it can name.

What it did not do is supply the five bands. Those are still typed on the command line, read off the specification's table by a person. The phrase the run prints for them — a `spec DECLARES` — is the band mechanism describing a human-supplied figure, not the reader describing one it extracted.

And this passage costs the document something. Every other figure here regenerates identically from its run. The text above does not: a language model produces different words each time, and the paragraph quotes words. What is stable is what surrounds it — the bands, the measurements, the verdicts — all of which were byte-for-byte identical across this run, two before it with different readings, and the one before those, when the reader declined for want of a key. The model changed what was *read*, not what was measured.

A declared band is not a measured one, and the run never reports it as one. Every message that uses a declared band says so in those words — a `spec DECLARES as 0.0000..3.3000 V, not measured` — and the blocks the open loop cannot reach say that too, rather than quietly characterising something else.

Per-block models are still not a system. That question does not go away with the co-simulation, so the loop is closed afterwards in *pure SystemVerilog*: the composed models, the design's own `pfd.sv` and `divn.sv` instantiated unchanged, one simulator. No DPI import, no bridge, no SPICE process anywhere in it.

What has to be stated is the *topology* — which RTL port lands on which net — because connectivity is not recoverable from a model, and on this circuit a name is actively misleading: in the reference example's own RTL, `d_R` is the *divided* clock, the opposite of what it reads like, and a loop wired from that name is inverted.

What is *not* stated is the sense of either pump gate. That follows from two things characterisation has already measured — the sign of the tuning curve and the direction of the current each control state drives — and the generated file carries the numbers it came from:

```
pdn <- up, active HIGH (derived)
ro_vco measured at 15 control points, 0.495 to 2.805 V, over which
its frequency FALLS 804.5 -> 16.74 MHz: Kvco is NEGATIVE
```

```

lpfilt: current into dra reaches vout through a state space whose
        input vector is non-negative throughout, so positive current
        RAISES it
with pupb quiescent, pdn at 1 drives -0.00447 A into dra and at 0
        drives -1.961e-09 A, both at 1.849 V -- so it is active HIGH

```

`pupb` comes out active *low* and `pdn` active *high* on the same pump. That is not a distinction a port name survives, and a charge pump wired from its names has shipped upside down before.

Co-simulation, at any stage	none
Models emitted, four routes	5
Operating bands	5 declared, 0 measured
Per-block equivalence	5 of 5 pass
Loop closed in	SystemVerilog only
Gate senses derived from measurement	2 of 2
Divider output vs the reference it follows against a 2% bar	10.000040 vs 10.000000 MHz +0.0004%, locked
Control voltage across the window drift end to end	1.8490–1.8507 V –0.16 mV
Environments generated blocks refused, with a reason run, and reporting	3 per block, 1 over the loop 2 4 of 4
Characterisation	33 min
Closing the loop and running it	37.3 s

And a UVM-MS environment over the whole loop. The loop above is a plain testbench: it drives a reference and checks a rate. What the same run also emits is a full environment around it — sequence, sequencer, driver, monitor, agent, scoreboard, and an MS proxy split into an abstract half the components talk to and a concrete half in the top level, so neither driver nor monitor holds a hierarchical path into the design.

It is the only *system* environment this tool generates with every one of those, and the reason is the stimulus:

environment	seq	drv	sqr	mon	agt	prx	scoreboard
block, transfer function	•	•	•	•	•	•	uvm_scoreboard
block, oscillator	•	•	•	•	•	•	uvm_scoreboard
system, co-simulated	—	—	—	•	•	•	uvm_scoreboard
system, no co-simulation	•	•	•	•	•	•	uvm_scoreboard

The co-simulated system environment has the observing half and none of the driving half, because the testbench it wraps brings its own stimulus — there is nothing for a sequence to ask for. Here there is no such testbench: the stimulus is the reference clock and this environment generates it, so a sequence, a sequencer and a driver have work to do.

The oscillator’s block environment is built on the same pattern, and the pattern is what makes it extensible. The timing lives in the proxy’s concrete half in `top`, which is the only scope that can see the design’s signals; the driver, monitor, agent, scoreboard and test hold none — zero references to either signal, against eight in `top`. A second observer can subscribe to the monitor’s analysis port without restructuring anything, and *one* implementation of “time a clock” serves this environment and the loop one.

What the loop environment’s sequence and driver do with that timing is step the reference by 0.5% and back, which is why the run below has three windows rather than one settled snapshot — a model frozen at its initial condition would produce that snapshot too.

Its goldens come from two places, and every line of its report says which. Two are the specification's: reach the reference within tolerance, keep the control voltage inside the stated window. Two are this run's own measurements, and the second of those is the one worth having. The oscillator's tuning curve was measured point by point in ngspice before any of this existed; inverting it at the frequency the loop is asked for names the control voltage that frequency lives at.

Window	divider output	control voltage
nominal, 10.000000 MHz	10.000040	1.8490–1.8507 V
stepped, 10.050000 MHz	10.049121	1.8437–1.8459 V
returned, 10.000000 MHz	9.999296	1.8493–1.8508 V
rate, against a 2% bar	3 checks, 0 failed, worst +0.0087%	[SPEC]
control window, 1.75–1.95 V	3 checks, 0 failed	[SPEC]
against the measured curve	3 checks, 0 failed, worst 0.90 mV	[MEASURED]
boundary finite	3 checks, 0 failed	[MEASURED]

The stepped window is the one to read. The reference went *up* and the control voltage went *down*, which is what a negative K_{vco} requires, and it went down by the amount the measured curve predicts. That curve puts 400 MHz at 1.8496 V from its 15 points, with $\pm 2\%$ spanning 1.8291–1.8705 V; the loop settles 0.90 mV from the prediction inside a window 41.5 mV wide.

That check can fail, which is the only reason to report it. A loop can reach the right rate at the wrong control voltage — through a mis-signed pump, a mis-fitted slope, a divider ratio the reference happens to absorb — and counting edges cannot see it. Shifting the measured table by 100 mV turns exactly those three checks red, worst 100.18 mV, while the two specification checks stay green. A specification-only environment would have passed the same run.

Three per-block environments are emitted beside it, for the two clock buffers and the oscillator, and two blocks are refused with the reason written out: the charge pump's verdict is a current per control state and the filter's a voltage from an injected current, and every generated section scores a voltage against a stimulus.

All four are then run, which is a different claim from all four being generated. A file count says a generator produced something; it says nothing about whether what it produced elaborates, compiles against the Accellera library, or scores anything. Each is executed and its scoreboard reported:

Environment	what its scoreboard concluded	time
inv1	54 DC checks, 0 failed, worst 0.001 mV of a 6.6 mV bar	5 s
inv2	54 DC checks, 0 failed	116 s
ro_vco	15 breakpoint + 5 held-out, 0 failed, worst 0.024%	203 s
the loop	12 checks over 3 windows, 0 failed	335 s

The two inverter figures are the weakest of these and the paragraph after next says why. All four environments are published, and all four were run from the published copy rather than from the run directory, so what is published is what ran.

What this result does not carry. Two things, and neither is small.

There are no transistors in this loop, so nothing here is evidence about the circuit. A loop that settles somewhere plausible and wrong still settles, and only the co-simulation above can catch that. The verdict covers this loop, at this reference, from the initial condition the deck states.

And the 5 of 5 is not the same 5 of 5 as the co-simulated run's. The duty metric that makes a clock buffer's verdict mean anything needs a clock *rate*, and the probe is what measures it. With no probe there is no rate, so `inv1` and `inv2` fall back to the average-voltage comparison — 0.021 and 0.026 against a 0.15 bar, green, and blind to edge placement for exactly the reason Section 6.3 gives. The two runs agree on the count and differ on what the count is worth.

7 Results at a glance

Case study	Environment	Checks	Failed
Filter	DC + AC	49 + 15	0
Duty corrector	DC + duty	46 + 14	0
Phase interpolator	phase per code	9	0
Phase-locked loop	co-simulation vs transistors	5 + 4	0
the same loop, no co-simulation	UVM-MS loop env, declared bands	5 + 12	0

The first three case studies — characterisation, model generation, both simulators, and every generated environment — run end to end in under 16 minutes on a laptop. The PLL is a further 15 minutes, most of it the ring oscillator's tuning curve. The no-co-simulation variant costs the same kind of time and for the same reason — 33 minutes on the run these figures come from, and it moves between runs because the oscillator's refinement passes do and because the machine is shared — of which closing the loop and running it is 37.3 seconds. Its extra twelve are the loop environment's, over three windows: rate and control window from the specification, agreement with the measured tuning curve and finiteness from this run's own measurements. Its 5 per-block passes are worth less than the row above them, for the reason given in Section 6.4.

Each is one command: `demo_4_pll.sh` for the co-simulated PLL, `demo_4b_pll_nocosim.sh` for the row beneath it. The co-simulated one differs from the others in what it needs, not in how it is run. Being the analog half of a co-simulation, it needs the other half: the RTL, the DPI bridge, and a *shared* libngspice — a different ngspice build from the binary the first three use. It checks for each of those before doing any work and names what is missing. A machine without them gets a skip saying which piece and where it comes from, rather than a failure twelve minutes into a run. `spice2rnm`'s outputs are simulator-agnostic: plain IEEE 1800 SystemVerilog, tied to no vendor. So that anyone can reproduce the results here, all of them were produced with open-source simulators — **ngspice** for characterisation and the reference transients, **xezim** — an open-source IEEE 1800 simulator that runs the Accellera UVM library¹ — for both the transient equivalence co-simulation and the generated UVM-MS environments. No commercial licence was used anywhere in this document. The generated artifacts themselves are published for inspection: the models, the complete environments, and the evidence files, exactly as the tool wrote them. They are at

<https://github.com/vrajeshprakhya/spice2rnm-demos>

under `showcase/`, stamped with the revisions of the tool, the demos and the simulator that produced them. What the repository holds:

```
spice2rnm-demos/
  README.md          what this repository is, and is not
  .gitignore         run scratch; the showcase is curated, not
                    whatever a run left behind
  demo_1_lpf.sh      demo_2_dcc.sh  demo_3_pi.sh
  demo_4_pll.sh      demo_4b_pll_nocosim.sh  run_all.sh
                    the five case studies, each one command;
```

¹<https://github.com/vrajeshprakhya/xezim>

```

run_all.sh chains the first four
netlists/      the four SPICE inputs, exactly as the demos
                run them: rc_lpf2, dcc, pi_therm, and the PLL
                deck with the supply-noise card it runs under
specs/         the three specifications in English: the
                filter's, the duty-cycle corrector's (demo 2
                reads it) and the PLL's (demo 4b reads it)
harness/       9 files: 8 measurement scripts the demos
                call, plus the JSON stating how the PLL's
                loop is wired
showcase/      the generated artifacts, byte-for-byte
lpf2/          the baseline case, 13 files + equivalence/:
  rc_lpf2_rnm.sv      the model, wreal ports
  rc_lpf2_ms_types_pkg.sv nettype real rc_lpf2_wire,
                      resolved by summing its drivers
  rc_lpf2_dms.sv      interconnect ports, straight through
  rc_lpf2_bridge*.sv  MS bridge and its driving core
  rc_lpf2_proxy_pkg.sv agent <-> bridge API
  rc_lpf2_ms_pkg.sv   item, sequence, driver, monitor,
                      agent, and the SPICE goldens
  rc_lpf2_ms_scoreboard.svh the checks and tolerances
  rc_lpf2_ms_tb/test.svh environment and test
  top_rc_lpf2_ms.sv   nets, DUT, bridge, test
  run_rc_lpf2_ms.sh   compiles and runs it
  result.json         fit, verdict, warnings
  equivalence/        what the check actually ran and found
dcc2/          same shape, 14 files + equivalence/: no AC
                section, adds duty goldens in dcc_ms_pkg.sv
                and the run log pipeline.txt
pi/            same shape, 13 files + equivalence/: an MS
                bridge, proxy and agent like the others, with
                phase-per-code goldens and no DC or AC section
pll/          53 files + 9 under equivalence/. Case study 4,
                checked against the transistors:
  11 at the top    five block models, three interfaces, the
                composed analog top, the node package,
                result.json
  env/inv1/ 12     and env/inv2/ 12 -- a full UVM-MS environment
  env/inv2/ 12     each: item, sequence, sequencer, driver,
                monitor, agent, scoreboard, env, test, MS
                proxy and bridge
  env/ro_vco/ 10   the oscillator's, same components, goldens a
                measured tuning curve rather than a DC sweep
  env/cosim/ 8     the SYSTEM environment: the design's own RTL
                and every block's model elaborate together,
                goldens the transistor run's own statistics,
                carrying a period-jitter monitor at 1 fs
pll_nocosim/    58 files + 9 under equivalence/. The same
                circuit with no co-simulation anywhere:
  11 at the top    the same models, interfaces and composed top
  env/inv1/ 12     env/inv2/ 12, env/ro_vco/ 10 -- as above
  env/rtl_loop/ 7   the LOOP environment. The only SYSTEM
                environment generated with the stimulus half,
                because it owns its stimulus: proxy package,
                package, scoreboard, env, test, top, run script
  loop/          2   the plain harness it replaces: one SystemVerilog
                file and the script that runs it
  rtl/           4   the customer's pfd.sv and divn.sv, the English
                specification, and the loop's topology in JSON
house_lib/      4 files, NOT generated: ng_ams_pkg.sv
                declaring nettype real ng_anet, and two
                hand-written models using it
lpf2_house/     13 files: lpf2 again, joined to house_lib --

```

	no types package, and rc_lpf2_rnm_ng_anet.sv
	in place of the wreal boundary
STAMP	per case: when it was curated, from which run,
	at which tool and simulator revisions; and the
	last full run's summary
refresh.sh	regenerates everything; the only way this
	directory is ever updated
whitepaper/	this document, source and PDF

7.1 What the equivalence folders hold

Every case study publishes an `equivalence/` directory: what the check ran, what it found, and a manifest of what was left out.

Case	Judged on	Published verdict
Filter	rms voltage	0.0290 against 0.15
Duty corrector	rms voltage	0.0769 against 0.15
Phase interpolator	phase per code	worst 1.832 ps at code 2, tolerance 6.251 ps
Phase-locked loop	five routes, then the whole system	five block verdicts, and the system over 116,018 samples

Start with two files. `equivalence/verdict.txt` is the summary of the check performed and what it concluded, one line per route: the metric, the worst point, the bar, and the verdict. The case's `result.json` beside it is the full record that line was taken from — what was driven, what was compared, over how many points, and every warning raised on the way.

Each folder carries three things. `verdict.txt` is lifted from that run's own `result.json` rather than retyped, so a number here cannot disagree with the run that produced it. The simulator and SPICE logs say what actually executed. And the comparison table gives one row per measured point. The phase interpolator's `phase_compare.csv` is nine rows, one per control code, with the measured delay, the model's delay and the error. The PLL's `rc_compare.csv` is the loop filter's response to an injected current, sample by sample.

What is manifested rather than published, and why. The waveform routes write one row per simulator timestep rather than one per measured point, so the filter's aligned comparison is 18.6 MB and its generated testbench 18 MB; the duty corrector's is 2.8 MB. The PLL is the weight: its golden co-simulation raw file alone is 202.8 MB, two thirds of everything the five cases produce. Close to 296 MB across them, in a repository whose history is permanent, for files `refresh.sh` regenerates in half an hour. Every one is named in `MANIFEST.txt` with its exact size and the command that recreates it, so a reader sees what exists and what it weighs without every future clone carrying it. What is published comes to 1.4 MB, of which the no-co-simulation PLL is 546 kB: four environments, the loop harness, the customer's RTL and both specifications.

The distinction is drawn by size rather than by filename. A comparison table small enough to read is evidence; one large enough to be a waveform is a trace. The routes differ in which they produce — a network compared point by point writes a table, a waveform compared sample by sample writes a trace — so the rule has to be about the file, not about what it is called. And what it deliberately does not hold: the `spice2rnm` tool itself (the product is licensed separately; this repository is its evidence), and the build products of the runs — characterisation scratch, compiled simulator objects, DPI shims. `refresh.sh` filters those out, so the showcase is what a user would keep, not everything a run produces.

8 What the equivalence check compares

Every model in this document was checked the same way. The generated SystemVerilog is loaded into a digital simulator and fed an input signal. The transistor circuit is fed the very same input in SPICE. The two outputs are then lined up in time and compared point by point, and both traces are saved, so anyone can go back to the run's own folder and work out the number again for themselves.

Two things about that arrangement matter more than the numbers it produces. The first is that **it reads the file you are actually handed**. There is a weaker number some tools report — how well the fitted maths matched its own measurements. That number never opens the generated file, and would look identical if the file had been written out wrongly. This check loads that file into a simulator and runs it, so a code-generation bug fails it.

The second is that **each block is scored on its own job**. One error figure used everywhere would be blind on most of these blocks: an average voltage error cannot see where an edge lands, and a frequency has no voltage error to measure in the first place.

If the block	then compare	and score it as
passes a signal through	the output waveform, on a sweep and on a second, unrelated input	the error, against how far the output itself moves
drives a clock	where the edges land, at the rate measured in use	duty error, in percentage points
moves an edge per setting	the crossing time at every setting	picoseconds, plus step size and evenness
oscillates	the output frequency, as the control voltage varies	a percentage of that frequency
switches a current	the current it drives, in each switch position	a percentage, for each position
is passive R/C	the output's response to a current pushed into it	the error, against how far the output moves

Why the fourth row cannot work like the first — two oscillators at the same frequency drift apart in phase, so subtracting their waveforms scores a perfect model as entirely wrong — is in Appendix A.

8.1 What keeps a figure from flattering itself

Some measurements are held back. Where a block is characterised by measuring it at many points, a share of those points is kept out of the model-building and used only for scoring. The headline figure is then not the fit marking its own work. The charge pump in case study 4 reports 0.549% held back; the ring oscillator reports its held-back frequency error directly.

An input the model has never seen. The filter in case study 1 is built from a frequency sweep and then tested with a pseudo-random bit sequence it was never shown (0.0097). A model that had merely memorised what it was built from is exposed there.

The report says what range the answer covers. A comparison proves nothing about conditions it never drove, so the report states the range it did drive. The ring oscillator is the clearest case, and the honest answer there is two numbers. Inside the voltage band the loop actually operates in, its held-back frequency error is 0.002%. At the far edge of the wider range that was characterised — where the curve bends hardest, and where the loop never goes — it reaches 4.99%. Both are reported. The verdict is taken in the band that matters, and the less flattering number is printed rather than quietly dropped.

What the model shape cannot do is priced in advance. Sometimes the shape of model chosen simply cannot represent something the circuit does. When that happens, the cost is stated up front, in the same units as the verdict — as with the inverter timing asymmetry in case study 4, quoted as duty error at the clock rate in use. A pass is then read against that floor, instead of being taken as a clean win.

8.2 Blocks, and then the system

The table above scores blocks one at a time. Putting blocks together asks a further question, because errors that are small individually need not stay small once everything is wired up. Three levels of check answer it.

The first needs nothing but the netlist. The assembled model is driven at the outside edge of the design, and its output is compared against the full transistor circuit driven the same way.

The second applies when what you hand the tool is a co-simulation — a design whose analog half already talks to a digital half — and it is the one case study 4 reports first. Here the design's own testbench is run twice. Once, the analog half is the generated models. Once, it is the real transistors. Everything else is left alone: the digital logic, the limits it checks against, and the pass/fail rules are byte-for-byte identical in both runs, and only the small piece of code joining the two halves is swapped. That makes the two runs directly comparable, signal by signal: 4 signals crossing between the halves, 0 failed, the control voltage agreeing to 4.15 mV on a rail carrying 20 mV rms of declared noise, and both versions locking to the same frequency.

The third is for the case that has no co-simulation to hand, which is most of them: an analog netlist, some RTL, and nothing joining them. The models are composed and the customer's own RTL is wired around them in SystemVerilog alone — no bridge, no SPICE in the loop — and the question asked is whether the assembled thing does what it is for. A full UVM-MS environment drives it, and judges it against the specification *and* against what this run measured: the oscillator's own tuning curve says which control voltage the target frequency lives at, so a loop reaching the right rate by the wrong route is caught rather than counted as a pass. Section 6.4 runs the same PLL that way and it locks to +0.0004% of its reference, 0.90 mV from where the measurement puts it. That is still a weaker claim than the second check makes, deliberately: there are no transistors in it to disagree with.

And a check that does not apply to a given run is reported as not applicable, with the reason why. It is never quietly counted as a pass.

8.3 One thing a score at an operating point cannot see

Every check described so far is *static*: it compares a value at an operating point. That is the right shape for most of what a model claims, and it is blind to one error that matters.

An oscillator model has to decide when to place its next edge. One way reads the control voltage at the instant the previous edge fired and schedules a whole half cycle from it. Another integrates the control across the cycle, which is what the circuit does. Both models carry the *same* tuning curve, so a check that measures frequency at held control voltages scores them identically — and it does: run against both, it passes both. Give them the same moving control and they part: on the reference ring, fed a control with a 100 ps correlation time, the sampling form produced $3.37\times$ the period jitter the transistors did.

How far apart they land is not a fixed property of the two forms. It is $\sqrt{T_{\text{half}}/\tau}$ in the correlation time τ of the disturbance, which for 1.25 ns against 100 ps predicts 3.54 — the measurement above, to within 5%. A faster disturbance separates them further, so a ratio quoted without its τ says less than it appears to.

So the tool checks the dynamic behaviour directly. A known disturbance is injected in SPICE at several amplitudes, and the disturbance *ngspice actually solved* is replayed into the model sample for sample. That last part is the point: giving the model its own random noise of

the same size would compare two random sequences, and agreement between those says more about how long the run was than about the model. Replaying the same waveform compares transfer.

Both of the model's inputs are exercised, not just the obvious one. On the reference ring the supply moves the frequency by 450.6 MHz/V, comparable to the control port and opposite in sign, so a model given no supply input cannot show what a droop does at all. The verdict is the worse of the two paths.

Path	Injected	Circuit	Model	Ratio
either	none (floor)	0.051 ps	0.000 ps	not judged
control	5 mV rms	2.396 ps	2.352 ps	0.981
	20 mV rms	8.743 ps	8.690 ps	0.994
	50 mV rms	26.719 ps	25.618 ps	0.959
supply	5 mV rms	2.808 ps	2.640 ps	0.940
	20 mV rms	11.617 ps	12.139 ps	1.045
	50 mV rms	28.422 ps	27.656 ps	0.973

These ratios are a draw, like the boundary figures earlier. Each injection is a fresh realisation and an rms over two hundred cycles carries about 5% on each side, so the same six points on the previous run read 0.987, 0.946, 0.925 and 0.896, 0.953, 0.970. That spread is why the bar is 0.25 rather than something that looks stricter: a tolerance tighter than the measurement's own uncertainty fails correct models at a rate set by run length.

The floor row is measured and deliberately not judged. With nothing injected the reference still reports about 54 fs, which is how precisely the edge-finder can locate a crossing rather than anything the circuit did. Asking the model to reproduce it would be asking it to reproduce an artefact. It is used instead as the floor every other amplitude has to clear before its result counts.

That agreement was not obtained by fitting anything. Take the supply path, where the sensitivity is separately measured: 450.6 MHz/V on a 2.509 ns period predicts 14.18 ps of period jitter for 5 mV rms, if the disturbance held still across the cycle. The circuit gives 2.797 ps — a ratio of 0.197, against the $\sqrt{\tau/T} = \sqrt{100.3 \text{ ps}/2509 \text{ ps}} = 0.200$ that averaging over the cycle predicts.

It holds across the decade rather than at one point: 0.197, 0.223 and 0.203 at the three judged amplitudes, where an oscillator that did not average its input would sit at 1 — five times higher. Three points within 12% of a parameter-free prediction, over a tenfold range, and each carrying the same few per cent of sampling uncertainty as the ratios above. The model reproduces it because it integrates, and the version that samples does not.

And it decides for itself whether the check is worth running, because for most designs the honest answer is no and a quarter of an hour is a real cost. Two things can say yes.

The first is the design's own closed-loop behaviour, measured: how far the control moves within one oscillator period, carried through the tuning curve's own slope into period movement, against what the reference measurement can resolve. On this PLL it says no, and says it with numbers — on a run where it was the deciding voice, 12.18 μV rms per 2.503 ns period, worth 0.0288 ps against a 0.0521 ps floor, 0.55 $\times$ what the reference can resolve. A loop filter holds its control still; that is what a loop filter is for.

The second is the netlist, and it is what runs the sweep above: this deck declares 20 mV rms on the ring's supply, and a design that writes noise into its own deck is asking for noise to be simulated. That rule is matched on the net rather than the source name, because block extraction rebuilds every supply as a plain DC source: the fragment carries `VBIAS2 dd 0 DC 3.3` where the top level wrote `vdd dd 0 dc {vcc} trnoise(...)`. The net survives extraction and the source name does not.

There is no flag to force it, because a request to run a check is not evidence that one is warranted. And where the tool has no closed-loop evidence either way, it runs the check rather

than skipping it: having no opinion about whether a check is needed is not the same as deciding it is not.

What the sweep measures, and what it does not. The ring in isolation, driven on its own characterisation fragment. That is the right scope for the question it answers — does this model transfer a disturbance correctly — and it is not the same question as what the loop does with one. Inside the loop the filter holds the control still, which is precisely why the closed-loop half of the gate declines; these numbers come from driving the block directly, where nothing is holding it. A reader should not read the supply column as a statement about this PLL’s supply rejection. It is a statement about the model.

9 Why a green testbench is worth believing

Generated verification has an obvious credibility problem. If the same tool writes the model *and* writes the test, what stops it from writing a test its own model is bound to pass? Three rules answer that.

The right answers come from SPICE, never from the model. Every expected value in every generated environment is a SPICE measurement of your circuit. Checking a model against the equations that model itself implements would pass no matter how wrong the model was, so spice2rnm never does it.

Every environment is proven able to fail. The tool deliberately breaks the generated model and confirms the test goes red. It also *measures* how small a break it can still catch, rather than simply claiming one. For the duty-cycle corrector’s environment:

Injected fault	Detected by
comparator offset, 20 mV	DC and duty sections
dynamics slowed 10×	duty section (DC is structurally blind)
dynamics slowed 50×	duty section (DC is structurally blind)

The right-hand column is the point. A DC check looks at the circuit after it has settled, and that is the whole of what many hand-written environments test. It cannot see *any* error in how fast a block responds — slow this model down fiftyfold and the DC check still passes it. The function-matched sections exist because of that blind spot, and the fault sweep is re-run against the shipped model so these detection claims stay true rather than becoming folklore.

The test is checked in both directions. A test that fails everything is as useless as one that passes everything. So the same sweep also confirms that errors small enough to be acceptable do *not* trip the environment.

10 Functional specifications as input

A netlist says what a circuit *is*. It never says what it is *for*, what rate it must run at, or how close is close enough. Hand those to the tool as a plain-language document — a datasheet excerpt, a requirements page — and it reads out the facts that shape verification:

From the specification	Used for
operating clock range	where the environment drives the block
accepted input duty range	the stimulus sweep
duty accuracy requirement	the pass/fail tolerance
block function	which environment is generated

Case study 2 is run this way, reading `specs/dcc_spec.md` from the repository: its duty environment checks 35–65% input duty at the specified clock rates against the specification’s own 1.0 pp requirement — 14 checks, 0 failed, worst 0.428 pp. **PASS**

The specification is cross-checked, never trusted. Every extracted fact is validated against what the circuit was actually measured to do, and the measurement wins. When the two disagree, that disagreement is surfaced as a finding:

```
SPEC CONFLICT: the spec says this block operates up to 4e+08 Hz, but
the fitted model is only valid below 1.195e+08 Hz -- 3.3x lower.
```

That is a coverage gap between a requirement and a characterisation, caught at model-generation time instead of at silicon correlation — and it is the kind of finding no amount of manual model review produces, because no one document contains both halves of it.

11 Joining an existing model library

A team with real-number models already has conventions — which user-defined nettype analog values travel on, which package declares it, what discipline their model ports use. Generated files that import their own freshly-invented nettype do not join that library; they sit beside it, each net scheme blind to the other, and the integration cost lands on the customer.

Point `--style-from` at the library and the output conforms: the environment’s analog nets use *their* nettype, its files import *their* package, no types package of the tool’s own is emitted, and the generated run script compiles their package file. A library whose models use `wreal` ports gets a model with `wreal` ports without being asked. Their timescale is reported but deliberately not adopted — generated delays are computed femtosecond values, and timescale is per compilation unit, so the directives interoperate unchanged.

Reading a house style is a judgement, so it is read and then verified. A library may declare several nettypes, only one of which is the living standard — and what says so is often a comment. Measured on a library that declares two, where the deprecated one happens to sort first alphabetically:

mechanical scan	picks the deprecated nettype, and warns
model reading	picks the live one, quoting the deprecation notice
verification	re-checks the choice against the named file

The model’s stated grounds, verbatim from that run: *“line 15 marks `aaa_legacy_wire` deprecated (“kept only so ancient testbenches still compile. Do NOT use in new code”), ... both models declare all analog ports as `acme_ewire`, confirming this is the live convention.”* Nothing in that reading is taken on trust: the named nettype, package and file are checked against the file before use, and a claim the file does not support leaves the mechanical answer standing, out loud. Where the tool has a *measurement* — a count of which discipline the library’s ports actually use — the count wins outright and a disagreement is recorded rather than adopted.

The result is an environment that runs on the house net, and it is published rather than described. The showcase carries the same filter twice: `lpf2/` generated normally, and `lpf2_house/` generated against `house_lib/` — a small hand-written library standing in for a customer’s own. Diffing the two directories is the whole argument:

	lpf2/	lpf2_house/
types package	emitted	not emitted
nets	rc_lpf2_wire	ng_anet
imports	the generated one	ng_ams_pkg
run script	compiles it	compiles the house package
model ports	plain real	ng_anet, and the environment instantiates it
the model	byte-identical	
the scoreboard	byte-identical	

Conformance changes the boundary and nothing else — and the published environment runs from where it sits, against the library beside it: 49 DC and 15 AC checks, 0 failed, exactly the default environment’s result.

Note what the environment instantiates in that second column. When the model is delivered behind a port boundary, the generated testbench drives *that* module rather than the core inside it, and the boundary’s file joins the compile list — so what the evidence covers is what ships, not a component of it. Measured separately, a library resolving its net by averaging rather than summing conforms and passes the same way, and a library using **wreal** ports gets **wreal** ports on the model instead. Without `--style-from`, or against a directory with nothing recognisable in it, nothing changes — and the run says so.

12 AI where it helps, physics where it decides

spice2rnm asks a language model four kinds of question: where one block ends and the next begins, what a block is for, what a written specification requires, and what conventions an existing model library follows (Section 11). Every one of those is a question of *intent*. None is a question of physics.

That is the rule the tool works by: **intent from the model, physics from the measurement**. Every suggestion is checked against the measured circuit before it is used. One that does not check out is discarded, the run carries on with what was measured, and it says that it did so. With no API key at all, every AI feature declines cleanly and the pipeline runs entirely on measurement.

Which of those two happened is recorded in the run’s own result, not only in its log. A verdict carries the metric it was reached by. A report read months later can then tell a check that ran from one that declined and fell back. Without that, every green result would have to be taken on trust that the feature requested was the feature exercised.

13 Scope and requirements

- **Inputs:** SPICE netlists, flat or with subcircuits; optional plain-language functional specification; optionally an existing RNM library to take conventions from.
- **Outputs:** SystemVerilog real-number models, with the port discipline of your choice — **wreal** by default, plain **real** where a flow has no **wreal** support, or your library’s own nettype; UVM-MS 1.0 environments; equivalence reports with archived waveforms.
- **Toolchain:** a SPICE simulator for characterisation, and a digital simulator capable of running real number models — with UVM support for the generated environments. The generated models and environments are standard SystemVerilog and carry no vendor dependence, so either may be the one your flow already uses.

- **Block-level scope:** single-input single-output blocks and compositions of them, including current-mode blocks sharing a node. Hierarchical composition trades some accuracy for structure; the equivalence report states the composed figure separately from the per-block figures, so the trade is visible, not hidden.
- **Jitter TRANSFER is verified; intrinsic device noise is characterised.** Both are measured, and they are measured differently.

What is checked is what the model does with a disturbance it is given: injected in SPICE, replayed into the model, period jitter compared, on both the control and the supply. Section 8.3 gives the measurements — six judged points over a tenfold range, worst ratio 0.940 against a 0.25 tolerance, reached by the pipeline’s own decision because the deck declares noise on a net the ring is wired to. This is a real check with a verdict, and it fails a model whose transfer is wrong.

The oscillator’s own noise is characterised through its impulse sensitivity function, which the pipeline measures without being asked: a narrow charge injected at each phase of the cycle, the lasting phase shift measured, and $\Gamma(\phi)$ is the sensitivity that results.² On the reference ring Γ_{rms} is 4.04×10^{12} rad/C from the automated sweep, against 4.33×10^{12} measured by hand: the two agree to 7%. Γ changes sign across the cycle as a ring’s must, and halves when the injected charge halves.

Γ_{rms} times a device noise density is a jitter: 0.236 to 0.578 ps rms here, or -114.5 to -106.7 dBc/Hz at 1 MHz, reported as a range because the channel-noise factor and the way per-node densities combine are both ranges. The model carries it as a parameter, shipped at zero: the figure is computed from the measured ISF rather than compared against a reference transient, since ngspice simulates no device noise in one.

Driven blocks jitter differently, and the models say so. An oscillator measures each edge from its own last one, so its phase random-walks and the error accumulates. A buffer or a lag block measures its output from an *input* event, so every traversal draws independently and nothing accumulates.³ The scheduled-lag models take a synchronous-jitter figure on that basis, perturbing the input-to-output delay once per traversal. That figure is supplied, not measured: a transient of a deck declaring no noise has no delay variation in it to measure. The generated file says so. It also states the constant latency the perturbation costs, because a tap only looks backwards, so an edge can be early only relative to a delay already present.

Edge JITTER is measured where the reference has edges. On the reference PLL’s two clock-path buffers it is 3.98 ps and 3.60 ps, reported beside the skew because the two fail differently. It is reported rather than judged: there is no threshold for it, and inventing one

²The method is Hajimiri, Limotyrakis and Lee, *Jitter and Phase Noise in Ring Oscillators*, IEEE JSSC 34(6), June 1999. Two details decide whether it works, and each returns a plausible wrong answer rather than an error, so the tool enforces both. The pulse is applied differentially where the circuit is differential — into one node of a differential ring it is mostly common mode, which the ring rejects by design, and the measurement then reads its own floor however large the pulse. And linearity is judged at the phase of greatest sensitivity rather than a convenient one: the same charge that moves the phase by 0.04 ps where the ring barely responds moves it by 52 ps at the peak.

³The distinction and both implementations are Kundert’s, in *Modeling and Simulation of Jitter in Phase-Locked Loops*, AACD 1997. He makes four points this model follows. Driven systems jitter without accumulating; autonomous ones have a phase difference that is “a random walk that is not bounded”. The metric for both is $J = \sigma(T)$, the standard deviation of one period. Driven jitter is modelled by “randomly dithering the delay argument ... on every output transition”. And the per-update dither scales by $\sqrt{N}$ for N updates per period, which is 1.414 for the two this model uses.

Each was checked. Asked for 1% rms period jitter, the model produced 0.9998%. Its scaling constant is the $\sqrt{2}$ his listing specifies. And over 30,000 cycles, the variance of the accumulated deviation grew linearly in cycle count, at 1.004 ± 0.026 of the kJ^2 his random walk requires.

One combination is refused: at one sub-step per half cycle the scheduler cannot resolve a randomised threshold, and the model declines it rather than report 39.6% jitter where 1% was asked.

from a single design would be worse than showing the number. The system equivalence check does not judge it either, and the reason is the probe rather than the model. Its boundary log is sampled at 100 ps. The composed model’s output crosses its threshold between samples, so a crossing can only be located to the nearest tick. Reporting that as jitter would be measuring the sampling grid.

The generated UVM-MS environment carries a monitor without that limitation. It is event-driven on the signal rather than sampled on a clock, and compiled at 1fs while the rest of the environment stays at 1ps, which costs nothing anywhere else because SystemVerilog scopes timescale per module. Running that environment on the composed PLL gives 12.70 ps rms of period jitter over 1151 periods at the boundary, with none of its system checks failed and one inconclusive, over 116,012 boundary crossings. The number of checks is not quoted because it is not fixed: one of them exists only where the transistor reference settled measurably, which under a noisy rail it may or may not do.

REPORTED, NOT JUDGED, and for a specific reason rather than caution. A loop suppresses its oscillator’s jitter inside the loop bandwidth, so a figure measured at the boundary is not the block’s own jitter and the two are not expected to agree. The 10.91 ps this deck’s declared rail is worth at `ro_vco` is a different measurement of a different quantity. On this run they differ by 16%; on the one before, by 3%. Both are draws, and neither is evidence about the other.

This is further than published practice goes on the validation side, while the gap named above — intrinsic noise in the model — is one the same practice also has. Analog Devices’ production RNM suite for a GHz transceiver validates its PLL on lock time and frequency error, the same quantities checked in Section 8. It states that “model validation remains an area for improvement in the real number modeling flow”, and names frequency-domain performance as future work.⁴

A What the routes measure

The case studies name the characterisation routes and say why each block needs the one it gets. This appendix says what the terms mean, using the reference PLL’s own measurements rather than an example invented for the purpose. None of it is needed to read the results; it is here for a reader who wants to check them.

A.1 Why an oscillator is compared as a rate

The first row of the table in Section 8 is the ordinary case: two output waveforms, lined up in time and subtracted. The fourth row cannot work that way, and the reason is worth a paragraph because it is the whole justification for the route.

Suppose the model and the transistors run at *exactly* the same frequency — a perfect model. They start aligned. Nothing locks their phase together, so a difference of one part in 10^5 slides them a full cycle apart within a millisecond, and from then on the model sits at one supply rail at the instants the circuit sits at the other. Subtract those two waveforms and you get full-swing error everywhere. The perfect model scores as entirely wrong.

That is not hypothetical, and a milder version of it is visible elsewhere in this document: the two clock buffers in case study 4 report a worst instantaneous error of 99.9% of full swing, from an edge landing slightly late. For a buffer that is a brief moment each cycle. For an oscillator it is the permanent steady state.

⁴J. Dudek, J. Neel and K. O’Donoghue (Analog Devices), *Real Number Modeling of RF Circuits*, DVCon proceedings, <https://dvcon-proceedings.org/wp-content/uploads/real-number-modeling-of-rf-circuits.pdf>. The paper notes that because RNMs exist to enable system-level testing, “the requirements for model validation are greatly relaxed in this environment”.

So no waveform is subtracted. The control voltage is set, the circuit is run in SPICE and the cycles counted, the model is run at the same control and its cycles counted, and the two frequencies are compared:

set <code>cont</code> to	1.9000 V
SPICE: count cycles, divide	f_{spice}
model: same control, count	f_{model}
score	$ f_{\text{model}} - f_{\text{spice}} /f_{\text{spice}} = 0.024\%$
against	a 2% bar

Phase is ignored deliberately. The model does not claim to reproduce it — it emits an ideal square wave, and the real ring’s finite edges were already accounted for when its frequency was measured by FFT.

Every verdict records which of these rows it came from. Months later, a reader can still tell which number actually decided the result from the numbers that were merely noted alongside it.

A.2 The tuning curve

A tuning curve is not a description of the model — it *is* the model. What the oscillator route measures is a table, one row per control voltage, each row a separate transistor-level transient in which the control is held and the resulting oscillation is timed:

<code>cont</code> (V)	frequency (MHz)
0.495	804.47
⋮	⋮
1.750	438.73
1.850	399.86
⋮	⋮
2.805	16.74

The generated model looks its frequency up in that table and interpolates between rows. That is also why its checks come in three kinds rather than one. At the table’s own 15 rows the model is exact by construction, so agreement there (0.001%) shows the table was transcribed and nothing more. At 5 control voltages *deliberately withheld* from it, the model must interpolate and is compared against fresh SPICE measurements — 0.024%, and the only one of the three that is evidence about accuracy. At 9 voltages far outside where the closed loop operates it reaches 4.987%, which is reported and not failed, because the loop never goes there.

It is the same table Section 6.4 reads backwards: at 400 MHz it says 1.8496 V, and that is the control voltage the assembled loop is then required to settle at.

A.3 Control states

A control state is one combination of the digital gates driving a block. The pump has two of them, `pdn` and `pupb`, and they are driven *independently*, so it has four states rather than one control axis — and one measured $I(V)$ table for each, 12 points from 0 to 3.3 V with 11 more held out:

state	(p _{dn} ,p _{up})	i at 1.80 V	at 3.3 V	what it is
00	(0, 0)	+7.239 mA	≈ 0	sources: charges the filter
01	(0, 1)	−1.9 nA	−4.0 nA	high-Z: off
10	(1, 0)	+2.798 mA	−5.078 mA	both on, fighting
11	(1, 1)	−4.441 mA	−5.078 mA	sinks: discharges it

Why a table per state and not one fitted curve. The gates are digital: each is on or off, so the block has no continuous control axis to fit against. It has a small finite set of modes, and within a mode its behaviour depends on the output voltage alone — how much current it can push into a node depends on what voltage is already there. That is what $I(V)$ means here, and why there are four of them.

It is also why the default route cannot express it, which the pipeline says from the topology rather than from the block’s name. An inverter ties its two gates together, so one input drives both and there is a single axis to fit. Here the two are separate digital signals, which makes 01 and 10 reachable — states an inverter does not have. 01 is the quiescent one, drawing nanoamps, and 10 is both devices conducting at once.

Those four rows are load-bearing twice more in this document. The loop harness of Section 6.4 derives each gate’s active level from them, read at the 1.849 V the loop sits at rather than at a table row: with **pupb** quiescent, **p_{dn}** at 1 reaches state 11 and sinks 4.47 mA, so it is active high; with **p_{dn}** quiescent, **pupb** at 0 reaches 00 and sources 7.103 mA, so it is active low. And the equivalence verdict names the state its worst error came from — 0.5487% against a 2% bar, at state 00 — because an average across four modes would hide which mode is the weak one.

A.4 The state space

A filter’s state space is what lets its route measure nothing at all. A state space is the ordinary way of writing a linear dynamic system — a few variables whose *rates of change* depend on their present values and on the input: $\dot{x} = Ax + Bu$, $y = Mx + Nu$. For an RC network the states are the capacitor voltages and nothing else. The deck says:

```
.subckt lpfilt dra vout
R1 dra int1 300 C1 c1 0 10n
R2 int1 c1 200 C2 int1 0 5n
R3 int1 vout 50 C3 vout 0 0.5n
.ends
```

Three capacitors, so three states: **c1**, **int1**, **vout**. The input is the current the pump injects into **dra**; the output is **vout**. “Exact and needs no fitting” then means every coefficient in the emitted model is a product of two numbers from those six lines:

emitted coefficient	is	which is
500,000	$1/(R_2 C_1) = 1/(200 \times 10 \text{ n})$	the c1 pole
40,000,000	$1/(R_3 C_3) = 1/(50 \times 0.5 \text{ n})$	the vout pole
1,000,000	$1/(R_2 C_2) = 1/(200 \times 5 \text{ n})$	c1 into int1
4,000,000	$1/(R_3 C_2) = 1/(50 \times 5 \text{ n})$	vout into int1
200,000,000	$1/C_2 = 1/5 \text{ n}$	the injected current

Not one of those came from a simulation. Compare the inverter in the same netlist, where the tool ran SPICE 33 times, fitted a curve, and reports an rms fit error: here there is nothing to fit and nothing to be wrong about, because Kirchhoff’s laws and the netlist determine the answer completely. What the equivalence check can still catch is a code-generation error — the emitted file disagreeing with the matrices it was built from — and that is what its 0.019% measures.

Two details of the emitted file follow from the same reasoning. `dra` is *not* a state, because it carries no capacitance: its voltage is whatever the resistors and the driven current imply at that instant, so it is solved algebraically each step together with the pump driving it. And the filter has no DC path to ground, so a constant injected current ramps the output forever without settling — which is exactly what a charge pump into a capacitive filter does, and why this is a state space rather than a DC gain.

© 2026 Veylon Systems. spice2rnm is a Veylon Systems product. Every figure in this document was produced by the shipped tool from its bundled demonstration circuits, most recently on 2026-09-22. The demonstration suite at <https://github.com/vrajeshprakhya/spice2rnm-demos> regenerates them in one command.

Veylon Systems